JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Class Clock

java.lang.Object
    java.time.Clock

```
public abstract class Clock
extends Object
```

A clock providing access to the current instant, date and time using a time-zone.

Instances of this class are used to find the current instant, which can be interpreted using the stored time-zone to find the current date and time. As such, a clock can be used instead of `System.currentTimeMillis()` and `TimeZone.getDefault()`.

Use of a `Clock` is optional. All key date-time classes also have a `now()` factory method that uses the system clock in the default time zone. The primary purpose of this abstraction is to allow alternate clocks to be plugged in as and when required. Applications use an object to obtain the current time rather than a static method. This can simplify testing.

Best practice for applications is to pass a `Clock` into any method that requires the current instant. A dependency injection framework is one way to achieve this:

```
public class MyBean {
  private Clock clock;  // dependency inject
  ...
  public void process(LocalDate eventDate) {
    if (eventDate.isBefore(LocalDate.now(clock)) {
      ...
    }
  }
}
```

This approach allows an alternate clock, such as `fixed` or `offset` to be used during testing.

The `system` factory methods provide clocks based on the best available system clock This may use `System.currentTimeMillis()`, or a higher resolution clock if one is available.

Implementation Requirements:

This abstract class must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe.

The principal methods are defined to allow the throwing of an exception. In normal use, no exceptions will be thrown, however one possible implementation would be to obtain the time from a central time server across the network. Obviously, in this case the lookup could fail, and so the method is permitted to throw an exception.

The returned instants from Clock work on a time-scale that ignores leap seconds, as described in `Instant`. If the implementation wraps a source that provides leap second information, then a mechanism should be used to "smooth" the leap second. The Java Time-Scale mandates the use of UTC-SLS, however clock implementations may choose how accurate they are with the time-scale so long as they document how they work. Implementations are therefore not required to actually perform the UTC-SLS slew or to otherwise be aware of leap seconds.

Implementations should implement Serializable wherever possible and must document whether or not they do support serialization.

Implementation Note:

The clock implementation provided here is based on `System.currentTimeMillis()`. That method provides little to no guarantee about the accuracy of the clock. Applications requiring a more accurate clock must implement this abstract class themselves using a different external clock, such as an NTP server.

Since:

1.8

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**`Clock`**()	Constructor accessible by subclasses.

Method Summary

Instance Methods | Abstract Methods | Concrete Methods

Modifier and Type	Method	Description
boolean	**equals**(**Object** obj)	Checks if this clock is equal to another clock.
static **Clock**	**fixed**(**Instant** fixedInstant, **ZoneId** zone)	Obtains a clock that always returns the same instant.
abstract **ZoneId**	**getZone**()	Gets the time-zone being used to create dates and times.
int	**hashCode**()	A hash code for this clock.
abstract **Instant**	**instant**()	Gets the current instant of the clock.
long	**millis**()	Gets the current millisecond instant of the clock.
static **Clock**	**offset**(**Clock** baseClock, **Duration** offsetDuration)	Obtains a clock that returns instants from the specified clock with the specified duration added
static **Clock**	**system**(**ZoneId** zone)	Obtains a clock that returns the current instant using best available system clock.
static **Clock**	**systemDefaultZone**()	Obtains a clock that returns the current instant using the best available system clock, converting to date and time using the default time-zone.

static **Clock**	**systemUTC**()	Obtains a clock that returns the current instant using the best available system clock, converting to date and time using the UTC time-zone.
static **Clock**	**tick**(**Clock** baseClock, **Duration** tickDuration)	Obtains a clock that returns instants from the specified clock truncated to the nearest occurrence of the specified duration.
static **Clock**	**tickMinutes**(**ZoneId** zone)	Obtains a clock that returns the current instant ticking in whole minutes using best available system clock.
static **Clock**	**tickSeconds**(**ZoneId** zone)	Obtains a clock that returns the current instant ticking in whole seconds using best available system clock.
abstract **Clock**	**withZone**(**ZoneId** zone)	Returns a copy of this clock with a different time-zone.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Clock

```
protected Clock()
```

Constructor accessible by subclasses.

Method Detail

systemUTC

```
public static Clock systemUTC()
```

Obtains a clock that returns the current instant using the best available system clock, converting to date and time using the UTC time-zone.

This clock, rather than `systemDefaultZone()`, should be used when you need the current instant without the date or time.

This clock is based on the best available system clock. This may use `System.currentTimeMillis()`, or a higher resolution clock if one is available.

Conversion from instant to date or time uses the UTC time-zone.

The returned implementation is immutable, thread-safe and `Serializable`. It is equivalent to `system(ZoneOffset.UTC)`.

Returns:

`a clock that uses the best available system clock in the UTC zone, not null`

systemDefaultZone

```
public static Clock systemDefaultZone()
```

Obtains a clock that returns the current instant using the best available system clock, converting to date and time using the default time-zone.

This clock is based on the best available system clock. This may use `System.currentTimeMillis()`, or a higher resolution clock if one is available.

Using this method hard codes a dependency to the default time-zone into your application. It is recommended to avoid this and use a specific time-zone whenever possible. The `UTC clock` should be used when you need the current instant without the date or time.

The returned implementation is immutable, thread-safe and `Serializable`. It is equivalent to `system(ZoneId.systemDefault())`.

Returns:

a clock that uses the best available system clock in the default zone, not null

See Also:

ZoneId.systemDefault()

system

public static Clock system(ZoneId zone)

Obtains a clock that returns the current instant using best available system clock.

This clock is based on the best available system clock. This may use System.currentTimeMillis(), or a higher resolution clock if one is available.

Conversion from instant to date or time uses the specified time-zone.

The returned implementation is immutable, thread-safe and Serializable.

Parameters:

zone - the time-zone to use to convert the instant to date-time, not null

Returns:

a clock that uses the best available system clock in the specified zone, not null

tickSeconds

public static Clock tickSeconds(ZoneId zone)

Obtains a clock that returns the current instant ticking in whole seconds using best available system clock.

This clock will always have the nano-of-second field set to zero. This ensures that the visible time ticks in whole seconds. The underlying clock is the best available system clock, equivalent to using system(ZoneId).

Implementations may use a caching strategy for performance reasons. As such, it is possible that the start of the second observed via this clock will be later than that observed directly via the underlying clock.

The returned implementation is immutable, thread-safe and Serializable. It is equivalent to tick(system(zone), Duration.ofSeconds(1)).

Parameters:

zone - the time-zone to use to convert the instant to date-time, not null

Returns:

a clock that ticks in whole seconds using the specified zone, not null

tickMinutes

```
public static Clock tickMinutes(ZoneId zone)
```

Obtains a clock that returns the current instant ticking in whole minutes using best available system clock.

This clock will always have the nano-of-second and second-of-minute fields set to zero. This ensures that the visible time ticks in whole minutes. The underlying clock is the best available system clock, equivalent to using `system(ZoneId)`.

Implementations may use a caching strategy for performance reasons. As such, it is possible that the start of the minute observed via this clock will be later than that observed directly via the underlying clock.

The returned implementation is immutable, thread-safe and `Serializable`. It is equivalent to `tick(system(zone), Duration.ofMinutes(1))`.

Parameters:

zone - the time-zone to use to convert the instant to date-time, not null

Returns:

a clock that ticks in whole minutes using the specified zone, not null

tick

```
public static Clock tick(Clock baseClock,
                         Duration tickDuration)
```

Obtains a clock that returns instants from the specified clock truncated to the nearest occurrence of the specified duration.

This clock will only tick as per the specified duration. Thus, if the duration is half a second, the clock will return instants truncated to the half second.

The tick duration must be positive. If it has a part smaller than a whole millisecond, then the whole duration must divide into one second without leaving a remainder. All normal tick durations will match these criteria, including any multiple of hours, minutes, seconds and milliseconds, and sensible nanosecond durations, such as 20ns, 250,000ns and 500,000ns.

A duration of zero or one nanosecond would have no truncation effect. Passing one of these will return the underlying clock.

Implementations may use a caching strategy for performance reasons. As such, it is possible that the start of the requested duration observed via this clock will be later than that observed directly via the underlying clock.

The returned implementation is immutable, thread-safe and `Serializable` providing that the base clock is.

Parameters:

`baseClock - the base clock to base the ticking clock on, not null`

`tickDuration - the duration of each visible tick, not negative, not null`

Returns:

`a clock that ticks in whole units of the duration, not null`

Throws:

`IllegalArgumentException - if the duration is negative, or has a part smaller than a whole millisecond such that the whole duration is not divisible into one second`

`ArithmeticException - if the duration is too large to be represented as nanos`

fixed

```
public static Clock fixed(Instant fixedInstant,
                          ZoneId zone)
```

Obtains a clock that always returns the same instant.

This clock simply returns the specified instant. As such, it is not a clock in the conventional sense. The main use case for this is in testing, where the fixed clock ensures tests are not dependent on the current clock.

The returned implementation is immutable, thread-safe and `Serializable`.

Parameters:

`fixedInstant - the instant to use as the clock, not null`

`zone - the time-zone to use to convert the instant to date-time, not null`

Returns:

`a clock that always returns the same instant, not null`

offset

```
public static Clock offset(Clock baseClock,
                           Duration offsetDuration)
```

Obtains a clock that returns instants from the specified clock with the specified duration added

This clock wraps another clock, returning instants that are later by the specified duration. If the duration is negative, the instants will be earlier than the current date and time. The main use case for this is to simulate running in the future or in the past.

A duration of zero would have no offsetting effect. Passing zero will return the underlying clock.

The returned implementation is immutable, thread-safe and `Serializable` providing that the base clock is.

Parameters:

`baseClock - the base clock to add the duration to, not null`

`offsetDuration - the duration to add, not null`

Returns:

`a clock based on the base clock with the duration added, not null`

getZone

```
public abstract ZoneId getZone()
```

Gets the time-zone being used to create dates and times.

A clock will typically obtain the current instant and then convert that to a date or time using a time-zone. This method returns the time-zone used.

Returns:

`the time-zone being used to interpret instants, not null`

withZone

```
public abstract Clock withZone(ZoneId zone)
```

Returns a copy of this clock with a different time-zone.

A clock will typically obtain the current instant and then convert that to a date or time using a time-zone. This method returns a clock with similar properties but using a different time-zone.

Parameters:

`zone - the time-zone to change to, not null`

Returns:

`a clock based on this clock with the specified time-zone, not null`

millis

```
public long millis()
```

Gets the current millisecond instant of the clock.

This returns the millisecond-based instant, measured from 1970-01-01T00:00Z (UTC). This is equivalent to the definition of `System.currentTimeMillis()`.

Most applications should avoid this method and use `Instant` to represent an instant on the time-line rather than a raw millisecond value. This method is provided to allow the use of the clock in high performance use cases where the creation of an object would be unacceptable.

The default implementation currently calls `instant()`.

Returns:

`the current millisecond instant from this clock, measured from the Java epoch of 1970-01-01T00:00Z (UTC), not null`

Throws:

`DateTimeException - if the instant cannot be obtained, not thrown by most implementations`

instant

```
public abstract Instant instant()
```

Gets the current instant of the clock.

This returns an instant representing the current instant as defined by the clock.

Returns:

`the current instant from this clock, not null`

Throws:

`DateTimeException - if the instant cannot be obtained, not thrown by most implementations`

equals

```
public boolean equals(Object obj)
```

Checks if this clock is equal to another clock.

Clocks should override this method to compare equals based on their state and to meet the contract of `Object.equals(java.lang.Object)`. If not overridden, the behavior is defined by `Object.equals(java.lang.Object)`

Overrides:

`equals` in class `Object`

Parameters:

`obj - the object to check, null returns false`

Returns:

`true if this is equal to the other clock`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this clock.

Clocks should override this method based on their state and to meet the contract of `Object.hashCode()`. If not overridden, the behavior is defined by `Object.hashCode()`

Overrides:

`hashCode` in class `Object`

Returns:

`a suitable hash code`

See Also:

```
Object.equals(java.lang.Object),
System.identityHashCode(java.lang.Object)
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.